

GUIA COMPLETO DE JAVA A PARTIR DO PROJETO PW2-INGRESSOS

POO • Java • Collections • Enum • Exceções • Maven • Spring Boot • HTTP • REST • JSON • JPA • ORM • Hibernate • JDBC • SQL Server • Lombok • Testes • Git

Este guia transforma o ZIP fornecido em uma apostila de estudo. A ideia é ensinar os conceitos necessários para entender o projeto inteiro, não apenas explicar o que cada arquivo faz.

Mapa mental: Java → POO → classes/objetos → coleções/enums/exceções → Maven → Spring Boot → HTTP/REST → Controller → JSON → JPA → Hibernate → JDBC → Driver → SQL Server.

O ZIP contém Maven, Spring Boot, uma entidade Filme, controllers, enums, configuração de banco, SQL, teste, documentação e um PDF de atividade. ■filecite■turn0file0■

1. JAVA: A BASE

Java é uma linguagem de programação orientada a objetos. O arquivo .java contém código-fonte; o compilador transforma esse código em bytecode; a JVM executa o bytecode. O JDK reúne as ferramentas usadas para desenvolver.

Conceito	Significado	Neste projeto
JDK	Kit de desenvolvimento Java.	O pom.xml define Java 25.
JVM	Máquina virtual que executa bytecode.	Executa o Spring Boot.
package	Organização de classes.	br.com.etechoracio.ingressos...
import	Importa classes de outros pacotes.	Spring, JPA, Lombok e utilitários.

Fundamentos indispensáveis: tipos, variáveis, operadores, if/else, switch, loops, métodos, arrays, String, escopo e conversões.

```
public class Exemplo {
    public static void main(String[] args) {
        int idade = 17;
        double nota = 8.5;
        String nome = "Matrix";

        if (nota >= 6) {
            System.out.println(nome + " aprovado");
        }
    }
}
```

Java é estaticamente tipado: uma variável tem um tipo. Tipos primitivos incluem int, long, double, boolean e char. String e classes são tipos de referência.

2. POO — PROGRAMAÇÃO ORIENTADA A OBJETOS

O projeto representa um filme como objeto. A classe Filme é o molde; cada Filme criado é um objeto. Atributos representam estado e métodos representam comportamento.

Conceito	Exemplo no projeto
Classe	Filme
Objeto	um filme específico criado em memória
Atributo	nome, ano, avaliacao
Método	listar(), cadastrar()
Encapsulamento	atributos privados + getters/setters
Abstração	Repository esconde detalhes de persistência
Polimorfismo	interfaces como List podem ter implementações diferentes

```
public class Filme {
    private String nome;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}

Filme f = new Filme();
f.setNome("Matrix");
```

Os quatro conceitos clássicos são **encapsulamento**, **abstração**, **herança** e **polimorfismo**. Mesmo que o projeto ainda não tenha uma hierarquia de classes, esses conceitos aparecem na forma como frameworks Java são projetados.

Construtor é executado quando um objeto é criado. O projeto usa Lombok para gerar um construtor com `@AllArgsConstructor` e um builder com `@Builder`.

3. COLLECTIONS, GENERICS E ENUM

Collections são estruturas para guardar grupos de objetos. O projeto usa `List<Long>` e `List<Filme>`. O `<Long>` é um generico: informa o tipo dos elementos.

Tipo	Uso
List	Sequência de elementos; pode repetir.
Set	Conjunto sem duplicidade lógica.
Map	Chave → valor.
ArrayList	Implementação comum de List.
HashSet	Implementação comum de Set.
HashMap	Implementação comum de Map.

```
List<Long> idsExistentes = List.of(1L, 2L, 3L);
```

```
if (idsExistentes.contains(id)) {  
    // o id existe  
}
```

```
return List.of(filme1, filme2);
```

Enum representa um conjunto fechado de valores. O projeto tem `CategoriaFilmeEnum`, `ClassificacaoIndicativaEnum` e `SimNaoEnum`. Isso evita strings soltas e reduz erros de digitação.

```
public enum SimNaoEnum {  
    S,  
    N  
}
```

4. EXCEÇÕES

Exceções representam problemas durante a execução. Aprenda `try`, `catch`, `finally`, `throw` e `throws`. Em uma API, uma exceção pode virar uma resposta HTTP adequada em vez de derrubar o cliente com uma resposta confusa.

```
try {  
    int n = Integer.parseInt("abc");  
} catch (NumberFormatException e) {  
    System.out.println("Valor inválido");  
}
```

Regra prática: não use `catch` vazio. O erro deve ser tratado, registrado ou convertido em uma resposta apropriada.

5. MAVEN E O pom.xml

Maven gerencia dependências e o processo de build. O `pom.xml` do ZIP usa Spring Boot, Java 25, Spring Data JPA, driver JDBC do SQL Server, Spring Web MVC e Lombok. `filecite turnOf file0`

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
    <parent>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-starter-parent</artifactId>  
        <version>4.1.0</version>  
        <relativePath/> <!-- lookup parent from repository -->  
    </parent>  
    <groupId>br.com.etechoracio</groupId>  
    <artifactId>pw2-ingressos</artifactId>  
    <version>0.0.1-SNAPSHOT</version>  
    <name/>  
    <description/>  
    <url/>  
    <licenses>  
        <license/>  
    </licenses>  
    <developers>  
        <developer/>  
    </developers>  
    <scm>  
        <connection/>
```

```

    <developerConnection/>
  </tag>
  <url/>
</scm>
<properties>
  <java.version>25</java.version>
</properties>
<dependencies>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <executions>
        <execution>
          <id>default-compile</id>
          <phase>compile</phase>
          <goals>
            <goal>compile</goal>
          </goals>
          <configuration>
            <annotationProcessorPaths>
              <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
              </path>
            </annotationProcessorPaths>
          </configuration>
        </execution>
        <execution>
          <id>default-testCompile</id>
          <phase>test-compile</phase>
          <goals>
            <goal>testCompile</goal>
          </goals>
          <configuration>
            <annotationProcessorPaths>
              <path>
                <groupId>org.projectlombok</groupId>
                <artifactId>lombok</artifactId>
              </path>
            </annotationProcessorPaths>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

```

```
    </plugin>
  </plugins>
</build>

</project>
```

Os comandos mais importantes são `mvn clean`, `mvn test`, `mvn package` e `mvn spring-boot:run`. O projeto também possui Maven Wrapper: `mvnw` e `mvnw.cmd`.

6. SPRING BOOT

Spring Boot simplifica a criação de aplicações Spring. Ele cria um contexto de aplicação, registra componentes e configura automaticamente várias partes com base nas dependências.

```
package br.com.etechoracio.ingressos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Pw2IngressosApplication {

    public static void main(String[] args) {
        SpringApplication.run(Pw2IngressosApplication.class, args);
    }

}
```

@SpringBootApplication é a anotação principal. SpringApplication.run inicializa a aplicação. Em uma aplicação web, o Spring Boot pode iniciar um servidor HTTP embutido.

Injeção de dependência: uma classe declara suas dependências e o Spring fornece os objetos. Isso evita acoplamento excessivo e facilita testes.

7. INTERNET, HTTP E API REST

Uma API é uma interface para comunicação entre sistemas. REST é um estilo arquitetural muito usado para APIs HTTP. O cliente envia uma requisição e o servidor devolve uma resposta.

Método	Uso	Rota do projeto
GET	Consultar	GET /filmes
GET + id	Consultar um recurso	GET /filmes/1
POST	Criar	POST /filmes
PUT	Atualizar	PUT /filmes/1
DELETE	Excluir	DELETE /filmes/1

Status essenciais: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found e 500 Internal Server Error.

REST trabalha com recursos. No projeto, o recurso principal é filme e a coleção é representada por /filmes.

8. JSON E O CAMINHO DE UMA REQUISIÇÃO

```
{
  "id": 1,
  "nome": "Matrix",
  "classificacao": "A16",
  "emCartaz": "S"
}
```

O Spring transforma JSON em objetos Java quando recebe @RequestBody e transforma objetos Java em JSON quando devolve um objeto no controller.

```
Cliente
  ↓ HTTP
Spring MVC
  ↓
Controller
  ↓
Service
  ↓
Repository / JPA
  ↓
Hibernate
  ↓
JDBC + Driver
  ↓
SQL Server
  ↑
```

JSON ← HTTP ← Cliente

9. FilmeController: ENTENDENDO O CÓDIGO REAL

```
package br.com.etechoracio.ingressos.controller;

import br.com.etechoracio.ingressos.entity.Filme;
import br.com.etechoracio.ingressos.enums.ClassificacaoIndicativaEnum;
import br.com.etechoracio.ingressos.enums.SimNaoEnum;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/filmes")
public class FilmeController {

    @GetMapping
    public List<Filme> listar(){
        Filme filme1 = Filme.builder()
            .id(1L)
            .nome("Matrix")
            .classificacao(ClassificacaoIndicativaEnum.A16)
            .emCartaz(SimNaoEnum.S)
            .build();
        Filme filme2 = Filme.builder()
            .id(2L)
            .nome("Homem aranha")
            .classificacao(ClassificacaoIndicativaEnum.A16)
            .emCartaz(SimNaoEnum.S)
            .build();
        return List.of(filme1, filme2);
    }
    List<Long> idsExistentes = List.of(1L, 2L, 3L);
    @GetMapping("/{id}")
    public ResponseEntity<Filme> buscarPorId(@PathVariable Long id){
        if(idsExistentes.contains(id)) {
            Filme filme1 = Filme.builder()
                .id(id)
                .nome("Matrix")
                .classificacao(ClassificacaoIndicativaEnum.A16)
                .emCartaz(SimNaoEnum.S)
                .build();
            return ResponseEntity.ok(filme1);
        }
        return ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Filme> cadastrar(@RequestBody Filme filme){
        filme.setId(100L);
        return ResponseEntity.status(HttpStatus.CREATED).body(filme);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Filme> atualizar(@RequestBody Filme filme,
        @PathVariable Long id){
        if(idsExistentes.contains(id)) {
            return ResponseEntity.ok(filme);
        }
        return ResponseEntity.notFound().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletar(@PathVariable Long id){
        if(idsExistentes.contains(id)) {
            return ResponseEntity.ok().build();
        }
        return ResponseEntity.notFound().build();
    }
}
```

@RestController marca a classe como controller REST. @RequestMapping("/filmes") define o prefixo. @GetMapping, @PostMapping, @PutMapping e @DeleteMapping ligam métodos Java aos métodos HTTP.

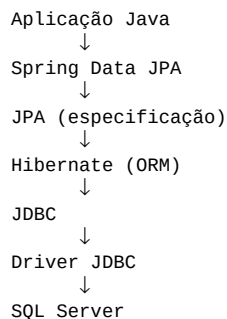
@PathVariable pega o valor da URL. @RequestBody transforma o JSON recebido em Filme. ResponseEntity permite controlar status e corpo.

No estado atual, o controller é didático: ele cria filmes em memória e usa idsExistentes fixos. Portanto, apesar de JPA e SQL Server estarem configurados, esses endpoints ainda não fazem um CRUD real no banco.

Isso é uma distinção essencial: configurar JPA não significa automaticamente que o controller esteja salvando dados. Para persistir, o controller precisa chegar a um repository/service.

10. JPA, ORM, HIBERNATE E JDBC

JPA é uma especificação/API de persistência. Hibernate é uma implementação de JPA e fornece ORM. JDBC é a API Java usada para comunicação com bancos relacionais. O driver JDBC adapta JDBC ao SGBD específico.



Camada	Responsabilidade
Spring Data JPA	Facilita repositórios e operações de persistência.
JPA	Define APIs/conceitos de persistência.
Hibernate	Implementa JPA e faz o mapeamento ORM.
JDBC	API de comunicação com banco relacional.
Driver	Implementação específica do SGBD.
SQL Server	Armazena os dados.

ORM: classe ↔ tabela; objeto ↔ linha; atributo ↔ coluna; id ↔ chave primária.

11. A ENTIDADE Filme

```
package br.com.etechoracio.ingressos.entity;

import br.com.etechoracio.ingressos.enums.CategoriaFilmeEnum;
import br.com.etechoracio.ingressos.enums.ClassificacaoIndicativaEnum;
import br.com.etechoracio.ingressos.enums.SimNaoEnum;
import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "TBL_FILME")
public class Filme {
    @Id
    @Column(name="ID_FILME")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name="TX_NOME")
    private String nome;

    @Column(name="NR_DURACAO")
    private Integer duracao;

    @Column(name="TP_CATEGORIA")
    @Enumerated(EnumType.STRING)
    private CategoriaFilmeEnum categoria;

    @Column(name="TP_CLASSIFICACAO")
    @Enumerated(EnumType.STRING)
    private ClassificacaoIndicativaEnum classificacao;

    @Column(name="NR_ANO")
    private Integer ano;

    @Column(name="TX_CAPA")
    private String capa;

    @Column(name="TX_DIRETOR")
    private String diretor;

    @Column(name="TX_ELENCO")
    private String elenco;

    @Column(name="TX_DESCRICAO")
    private String descricao;

    @Column(name="NR_AVALIACAO")
    private Double avaliacao;

    @Column(name="CHK_EM_CARTAZ")
    private SimNaoEnum emCartaz;

    @Transient
    private LocalDateTime dataExclusao;
}
```

`@Entity` diz que Filme é uma entidade persistente. `@Table(name="TBL_FILME")` liga a classe à tabela. `@Id` identifica a chave primária. `@Column` liga atributos às colunas. `@GeneratedValue` configura a geração do ID.

`@Enumerated(EnumType.STRING)` salva enums como texto. `@Transient` exclui `dataExclusao` da persistência.

Java	SQL
Filme	TBL_FILME
Long id	ID_FILME BIGINT
String nome	TX_NOME VARCHAR(50)
Integer duracao	NR_DURACAO INT
CategoriaFilmeEnum	TP_CATEGORIA VARCHAR(17)

Java	SQL
ClassificacaoIndicativaEnum	TP_CLASSIFICACAO VARCHAR(5)
SimNaoEnum	CHK_EM_CARTAZ VARCHAR(1)
Integer ano	NR_ANO INT
String capa/diretor/elenco/descricao	TX_CAPA/TX_DIRETOR/TX_ELENCO/TX_DESCRICAO
Double avaliacao	NR_AVALIACAO FLOAT
LocalDateTime dataExclusao	Não persiste

Lombok: @Getter gera getters; @Setter gera setters; @Builder permite construção fluente; @AllArgsConstructor gera construtor com todos os campos. Em entidades JPA, é comum também manter um construtor sem argumentos, dependendo do modelo adotado.

12. application.properties E CONEXÃO COM A WEB/BANCO

```
spring.application.name=pw2-ingressos
```

```
#DATASOURCE
spring.datasource.url=jdbc:sqlserver://localhost;databaseName=pw2_ingressos;encrypt=true;trustServerCertificate=true
spring.datasource.username=sa
spring.datasource.password=*123456HAS*
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
```

```
# JPA
spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.id.new_generator_mappings=false
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

spring.datasource.url aponta para o SQL Server local; username/password são credenciais; driver-class-name é o driver JDBC; o dialeto informa ao Hibernate como gerar SQL compatível.

ddl-auto=validate: o Hibernate valida o esquema em vez de criar/alterar automaticamente as tabelas.

show-sql=true: mostra SQL no console, ótimo para estudar.

Nunca deixe senhas reais versionadas em repositórios. Em projetos reais, use variáveis de ambiente ou secrets.

13. SQL SERVER E banco.txt

```
CREATE DATABASE pw2_ingressos;■
■
USE pw2_ingressos;■
■
CREATE TABLE TBL_FILME (■
    ID_FILME BIGINT PRIMARY KEY IDENTITY,■
    TX_NOME VARCHAR(50) NOT NULL,■
    NR_DURACAO INT NOT NULL,■
    TP_CATEGORIA VARCHAR(17) NOT NULL,■
    TP_CLASSIFICACAO VARCHAR(5),■
    CHK_EM_CARTAZ VARCHAR(1),■
    NR_ANO INT,■
    TX_CAPA VARCHAR(300),■
    TX_DIRETOR VARCHAR(300),■
    TX_ELENCO VARCHAR(300),■
    TX_DESCRICAO VARCHAR(500),■
    NR_AVALIACAO FLOAT■
);■
■
INSERT INTO TBL_FILME■
(TX_NOME, NR_DURACAO, TP_CATEGORIA, TP_CLASSIFICACAO, CHK_EM_CARTAZ, NR_ANO, TX_CAPA, TX_DIRETOR, TX_ELENCO, TX_DESCRICAO, NR_AVALIACAO)■
VALUES■
('Mad Max: Estrada da Fúria', 120, 'ACAO', 'A16', 'S', 2015, 'madmax.jpg', 'George Miller', 'Tom Hardy, Charlize Theron', 'Em um mundo pós-apocalíptico', 10),
('Clube da Luta', 139, 'DRAMA', 'A18', 'N', 1999, 'clubedaluta.jpg', 'David Fincher', 'Brad Pitt, Edward Norton', 'Um homem cria um clube secreto com regras próprias', 9),
('Homem-Aranha: Sem Volta Para Casa', 148, 'ACAO', 'A12', 'S', 2021, 'spiderman.jpg', 'Jon Watts', 'Tom Holland, Zendaya', 'Peter Parker enfrenta o vilão', 10),
('O Iluminado', 146, 'TERROR', 'A16', 'N', 1980, 'iluminado.jpg', 'Stanley Kubrick', 'Jack Nicholson, Shelley Duvall', 'Um homem enlouquece isolado', 10),
('Divertida Mente', 95, 'ANIMACAO', 'LIVRE', 'S', 2015, 'divertidamente.jpg', 'Pete Docter', 'Amy Poehler, Bill Hader', 'As emoções de uma garota', 10);
```

CREATE DATABASE cria o banco. USE seleciona o banco. CREATE TABLE cria a estrutura. PRIMARY KEY define a chave primária. IDENTITY gera IDs. NOT NULL impede valores ausentes. INSERT INTO insere dados.

O banco precisa combinar com o mapeamento da entidade. É por isso que nomes como ID_FILME, TX_NOME e NR_DURACAO aparecem tanto no SQL quanto em @Column.

14. REPOSITORY: A PONTE COM O BANCO

O ZIP não possui um FilmeRepository. Para transformar o projeto em CRUD real, uma interface típica seria:

```
public interface FilmeRepository
    extends JpaRepository<Filme, Long> {
}
```

Ao herdar JpaRepository, você recebe operações como findAll(), findById(), save(), deleteById() e existsById(). O Spring Data cria a implementação automaticamente.

15. SERVICE: REGRAS DE NEGÓCIO

```
@Service
public class FilmeService {

    private final FilmeRepository repository;

    public FilmeService(FilmeRepository repository) {
        this.repository = repository;
    }

    public List<Filme> listar() {
        return repository.findAll();
    }

    public Filme salvar(Filme filme) {
        return repository.save(filme);
    }
}
```

A divisão mais comum é Controller = HTTP; Service = regras de negócio; Repository = persistência. Isso deixa cada camada com uma responsabilidade clara.

16. CRUD COMPLETO: COMO PENSAR

Operação	Fluxo
GET /filmes	repository.findAll()
GET /filmes/{id}	repository.findById(id)
POST /filmes	repository.save(filme)
PUT /filmes/{id}	buscar → validar → alterar → save
DELETE /filmes/{id}	verificar → deleteById(id)

```
@GetMapping
public List<Filme> listar() {
    return repository.findAll();
}

@PostMapping
public ResponseEntity<Filme> cadastrar(
    @RequestBody Filme filme) {
    Filme salvo = repository.save(filme);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(salvo);
}
```

17. VALIDAÇÃO E ERROS

Uma API real precisa validar entrada. Jakarta Validation oferece `@NotBlank`, `@NotNull`, `@Size`, `@Min`, `@Max` etc. O controller pode usar `@Valid`.

```
@NotBlank
@Size(max = 50)
private String nome;

@Min(1888)
private Integer ano;

@PostMapping
public ResponseEntity<Filme> cadastrar(
    @Valid @RequestBody Filme filme) {
    ...
}
```

Para erros globais, `@RestControllerAdvice` permite transformar exceções em respostas HTTP padronizadas. Isso evita duplicar try/catch em todos os controllers.

18. TESTES

```
package br.com.etechoracio.ingressos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class Pw2IngressosApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

`@SpringBootTest` carrega o contexto da aplicação. `contextLoads` é um teste básico para verificar se a aplicação consegue iniciar.

19. SalaController E PARTES INCOMPLETAS

```
package br.com.etechoracio.ingressos.controller;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class SalaController {

    private List<Long> idsExistentes = new ArrayList<>(List.of(1L, 2L, 3L, 4L, 5L));

    private Long gerarIdAleatorio() {
        return new Random().nextLong(5) + 1;
    }

    // Implementação do seu código

}
```

SalaController ainda não é um controller REST completo. Ele possui uma lista de IDs e um método privado para gerar um número de 1 a 5. O comentário mostra que a implementação ficou pendente. Isso também faz parte do conteúdo do ZIP e é útil para perceber que um projeto pode estar em desenvolvimento.

20. GIT E A ATIVIDADE

O PDF de atividade incluído no ZIP pede fork, clone, branch com o nome da dupla, modelagem das classes, commit, pull request e nome da dupla no README. ■filecite■turn0file0■

```
git clone URL
cd PW2-ingresso-banco-main
git checkout -b nome-da-dupla
git status
git add .
git commit -m "Implementa modelagem"
git push -u origin nome-da-dupla
```

clone baixa o repositório; branch cria uma linha de trabalho; add prepara alterações; commit registra uma versão; push envia para o remoto; pull request propõe a integração.

21. CADA ARQUIVO DO ZIP

Arquivo	O que estudar
pom.xml	Maven, dependências, Java, Spring, Lombok, JDBC
application.properties	Configuração de banco e JPA
Pw2IngressosApplication.java	Inicialização do Spring Boot
entity/Filme.java	POO, JPA, ORM, annotations, enum
enums/*.java	Enum
controller/FilmeController.java	HTTP, REST, JSON, CRUD didático
controller/SalaController.java	Collections e código incompleto
banco.txt	SQL Server, DDL, DML
Pw2IngressosApplicationTests.java	Teste de contexto
docs/Atividade ingressos.pdf	Git, modelagem e entrega
.mvn/mvnw/mvnw.cmd	Maven Wrapper
README.md	Documentação

22. TRILHA PARA APRENDER JAVA DE VERDADE

Etapa	Aprenda
1	Sintaxe, tipos, operadores, if, loops e métodos
2	Classes, objetos, construtores e encapsulamento
3	Herança, interfaces, abstração e polimorfismo
4	Collections, generics, enum e exceptions
5	Maven e testes
6	Spring Boot e injeção de dependência
7	HTTP, REST e JSON
8	SQL e modelagem relacional
9	JPA, Hibernate e ORM
10	Controller → Service → Repository
11	DTO, validação, paginação e filtros
12	Segurança, logs, Docker, CI/CD e deploy

Checklist: ■ sei criar classes; ■ entendo objetos; ■ sei usar List/Set/Map; ■ sei usar enum; ■ entendo exceções; ■ entendo pom.xml; ■ entendo Spring; ■ entendo DI; ■ entendo HTTP/REST/JSON; ■ sei ler um controller; ■ sei explicar JPA/Hibernate/JDBC/driver; ■ sei mapear entidade para tabela; ■ sei criar repository; ■ entendo o caminho até o banco; ■ sei executar testes; ■ sei usar Git básico.

23. RESUMO PARA MEMORIZAR

Java = linguagem. **POO** = organização por objetos. **Maven** = build/dependências. **Spring Boot** = infraestrutura da aplicação. **HTTP** = comunicação web. **API** = interface entre sistemas. **REST** = estilo para APIs. **JSON** = formato de dados. **Controller** = HTTP. **Service** = regras. **Repository** = persistência. **JPA** = especificação. **Hibernate** = implementação ORM. **JDBC** = API de banco. **Driver** = ponte com o SGBD. **SQL Server** = banco.

Frase-chave: Cliente → HTTP → Controller → Service → Repository/JPA → Hibernate → JDBC → Driver → SQL Server → volta em JSON.

24. EXERCÍCIOS PROGRESSIVOS

1) Crie Cliente com id, nome e email. 2) Crie um enum TipoIngresso. 3) Crie uma List de filmes e filtre os em cartaz. 4) Busque filme por id. 5) Crie FilmeNaoEncontradoException. 6) Crie FilmeRepository. 7) Faça GET /filmes usar o banco. 8) Faça POST persistir. 9) Faça GET /filmes/{id} retornar 404. 10) Complete PUT e DELETE. 11) Adicione validação. 12) Escreva testes para service e controller.

A melhor forma de estudar este material é tentar explicar cada seta do fluxo e, depois, abrir o código do ZIP para localizar onde aquela ideia aparece. Assim você aprende o conceito e sua aplicação prática ao mesmo tempo.

Material baseado no conteúdo do ZIP fornecido, incluindo os arquivos Java, Maven, configuração, SQL e documentação da atividade.
■filecite■turn0file0■